UNIVERSITY OF INFORMATION TECHNOLOGY AND SCIENCES

Department of Computer Science and Engineering

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

CampusConnect

University Club, Event & Activity Management Platform

Course: Software Project Design and Development
Course Code: 416

SUBMITTED BY	SUBMITTED TO
Md. Shakhawat Hossain Student ID: 0432320005101009 Batch: 54	Al Imtiaz Course Teacher Software Project Design and Development

University of Information Technology and Sciences

Document Information

Item	Details
Document title	Software Requirements Specification (SRS) — CampusConnect
Project title	CampusConnect: University Club, Event & Activity Management Platform
Prepared by	Md. Shakhawat Hossain
Student ID / Batch	0432320005101009 / 54
Course / Code	Software Project Design and Development / 416
Course teacher	Al Imtiaz
Version	1.0 — Draft for course submission
Date	13 October 2026

Table of Contents

• 1. Introduction

• 2. Overall Description

• 3. Functional Requirements

• 4. External Interface Requirements

• 5. Non-functional Requirements

• 6. Data Requirements

• 7. System Design Diagrams

• 8. Use Cases and Workflows

• 9. Testing and Acceptance Criteria

• 10. Risks and Future Enhancements

• 11. AI-Assisted Revision and Justification

• Appendix A: Glossary

• Appendix B: References

This is a baseline SRS draft prepared from the project description provided. Follow any official course template if your teacher has supplied one. The final database choice and approval rules should be confirmed before implementation.

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements for CampusConnect, a centralized web platform for university clubs, events and student activities. It describes the expected system behaviour, user roles, interfaces, data and workflows so the project can be designed, implemented and tested consistently.

1.2 Scope

Students will discover campus activities, register for events, join clubs, view participation history and download issued certificates. Club executives will create and manage events, registrations, attendance, announcements and certificates. Faculty advisors and administrators will review event requests, supervise clubs and access activity reports.

The proposed stack is React.js, TypeScript and Tailwind CSS for the frontend; Node.js and Express.js for REST APIs; and PostgreSQL or MongoDB for persistence. JWT authentication, QR-code attendance, email notifications and PDF certificate generation are planned. Choose one database before implementation.

1.3 Objectives

• Centralize discovery of university events and clubs.

• Enable event registration and club membership requests.

• Support authorized participant and attendance management.

• Generate digital certificates for eligible attendees.

• Provide advisor/admin approvals and activity analytics.

• Reduce manual record keeping and improve communication.

1.4 Intended Audience

Student developer, course teacher, project teammates, testers, club organizers, faculty advisors and university administrators.

1.5 Definitions

Term	Definition
SRS	Software Requirements Specification
RBAC	Role-Based Access Control
API	Application Programming Interface
JWT	JSON Web Token
QR code	Machine-readable code used to identify a registration for attendance
CRUD	Create, Read, Update and Delete
UAT	User Acceptance Testing

1.6 Requirement Convention

Requirements are assigned identifiers such as FR-01 and NFR-01. “Shall” means mandatory. Priority labels are Must, Should or Could.

2. Overall Description

2.1 Product Perspective

CampusConnect is a standalone web application and is not assumed to replace the university's official student information system. It maintains its own club, event, registration, attendance, certificate, announcement and feedback records. Official single sign-on or student database integration is out of scope unless approved.

2.2 User Classes

User	Responsibilities	Access
Student	Browse events/clubs, register, request membership, view history/certificates	Own profile and own records
Club Member	Participate in club activities	Student permissions plus permitted member content
Club Executive	Manage assigned club, events, registrations, attendance and announcements	Assigned club(s) only
Faculty Advisor	Review event requests, supervise clubs, view reports	Assigned clubs
University Administrator	Manage users/clubs/categories and platform-wide reports	Administrative permissions

2.3 Operating Environment

Current Chrome, Edge, Firefox or Safari on desktop and mobile. React, TypeScript, Tailwind CSS and optional Framer Motion on the client; Node.js/Express REST API on the server; one selected database; email transport, QR library and PDF library as needed.

2.4 Constraints

• Protected API operations must enforce authentication and authorization server-side.

• Executives may manage only their assigned clubs and events.

• Capacity, deadlines and duplicate registrations must be checked on the server.

• Personal data should be limited to what the platform needs.

• Email delivery depends on a configured provider; QR attendance needs a camera/scanner.

• The interface should work at common desktop and mobile widths.

2.5 Assumptions and Dependencies

Users have internet access and a supported browser. Event approval requires an assigned advisor or administrator. Email notifications require working email configuration. University brand assets and policies will be supplied if needed.

2.6 Out of Scope for Initial Release

• Online payments, ticket sales or refunds.

• Native Android/iOS apps.

• Biometric attendance or location tracking.

• Official university SSO/SIS integration.

• Automated disciplinary decisions or public student rankings.

3. Functional Requirements

3.1 Account and Authentication

ID	Requirement	Priority
FR-01	The system shall allow users to register with required details and a valid email address.	Must
FR-02	The system shall authenticate users and establish a secure session/token after successful login.	Must
FR-03	Users shall be able to log out; sessions/tokens shall expire or be invalidated appropriately.	Must
FR-04	The system shall enforce role-based permissions for Student, Club Executive, Faculty Advisor and Administrator.	Must
FR-05	Users shall view and update permitted profile fields.	Must
FR-06	Client and server validation shall provide understandable error messages.	Must
FR-07	The system should provide account recovery through a verified channel.	Should
FR-08	Users shall not access another user's private records without authorization.	Must

3.2 Event Discovery and Registration

ID	Requirement	Priority
FR-09	Students shall browse published events and view description, organizer, venue/mode, date/time, category, capacity and deadline.	Must
FR-10	Students should search/filter events by keyword, category, date and club.	Should
FR-11	Students shall register while registration is open and capacity remains.	Must
FR-12	The system shall prevent duplicate active registrations by the same student for the same event.	Must
FR-13	Successful registration shall be confirmed and shown in the student's account.	Must
FR-14	Students should cancel registration when event rules permit it.	Should
FR-15	Students shall view upcoming registrations and participation history.	Must
FR-16	The system shall explain when an event is full, closed, cancelled or not yet open.	Must

3.3 Clubs and Membership

ID	Requirement	Priority
FR-17	Users shall browse active clubs and view their descriptions and categories.	Must
FR-18	Students shall submit membership requests when a club accepts members.	Must
FR-19	Executives shall approve/reject requests for their own club.	Must
FR-20	Membership status shall include Pending, Active, Rejected or Left.	Must
FR-21	Administrators should be able to create, deactivate and archive clubs.	Should

3.4 Event Creation and Approval

ID	Requirement	Priority
FR-22	Executives shall create and edit event drafts for clubs they manage.	Must
FR-23	Events shall have title, description, category, schedule, venue/online link, capacity, deadline and organizer.	Must
FR-24	Event proposals shall be submitted for approval before public publication.	Must
FR-25	Advisors/admins shall approve, reject or request changes and may record a decision note.	Must
FR-26	Authorized executives shall publish approved events and update permitted event statuses.	Must
FR-27	The system should notify registered participants of significant changes or cancellations.	Should
FR-28	The system shall validate event dates and registration deadlines.	Must

3.5 Attendance, Certificates and Announcements

ID	Requirement	Priority
FR-29	Authorized organizers shall view participant lists for their events.	Must
FR-30	The system should generate a unique QR token for an event registration.	Should
FR-31	Authorized organizers shall be able to record attendance manually as a fallback.	Must
FR-32	Invalid, duplicate or unrelated attendance tokens shall be rejected.	Must
FR-33	Authorized organizers shall generate PDF certificates for eligible attendees.	Must
FR-34	Students shall view/download certificates issued to their own accounts.	Must
FR-35	Executives shall publish club or event announcements.	Must
FR-36	The system should email registration, approval, event-change and certificate notifications.	Should
FR-37	The system could allow feedback after an event if enabled.	Could

3.6 Reports and Analytics

ID	Requirement	Priority
FR-38	Executives shall view registration and attendance totals for their own events.	Must
FR-39	Advisors/admins shall view activity summaries within their permission scope.	Must
FR-40	Reports should support date and category filters.	Should
FR-41	Authorized users should export reports as CSV or PDF.	Should
FR-42	Private student data shall not be exposed in public analytics.	Must

4. External Interface Requirements

4.1 User Interface

Main pages: Login/Register, Student Dashboard, Event Listing/Detail, Club Listing/Detail, My Registrations, My Certificates, Executive Dashboard, Event Editor, Attendance Management, Approval Queue and Admin Dashboard. The UI shall use clear navigation, consistent status labels, readable typography and responsive layouts.

4.2 REST API Interface

The frontend shall communicate with the backend over HTTPS in production using JSON. Protected endpoints shall require authentication. API errors should use consistent status codes and safe, understandable messages.

Resource	Example endpoints (illustrative)	Purpose
Authentication	POST /api/auth/register; POST /api/auth/login	Account access
Events	GET /api/events; GET /api/events/:id; POST /api/events	Discover/manage events
Registrations	POST /api/events/:id/registrations; GET /api/me/registrations	Register/view own records
Clubs	GET /api/clubs; POST /api/clubs/:id/membership-requests	Browse/join clubs
Attendance	POST /api/events/:id/attendance; GET /api/events/:id/attendees	Record/view attendance
Certificates	POST /api/events/:id/certificates; GET /api/me/certificates	Issue/download certificates
Reports	GET /api/reports/club/:id; GET /api/reports/events	Authorized analytics

4.3 Hardware and Software Interfaces

A modern browser and Node.js-compatible server are required. A phone camera or QR scanner may be used for attendance. The final README shall identify selected database, libraries and service configuration.

4.4 Communications

Production traffic shall use HTTPS. Secrets and API keys shall be stored in environment configuration and must not be committed to the repository.

5. Non-functional Requirements

ID	Requirement	Priority
NFR-01 Security	Passwords shall be stored using a modern password hash, never plain text.	Must
NFR-02 Authorization	Every protected operation shall check role and resource ownership on the server.	Must
NFR-03 Transport	Production traffic shall use HTTPS; sensitive tokens shall not be logged or placed in URLs.	Must
NFR-04 Privacy	Only necessary personal data shall be collected and private records restricted.	Must
NFR-05 Performance	Common list/detail requests should respond within 2 seconds at the 95th percentile under modest course-demo load, excluding email delays.	Should
NFR-06 Usability	Students should be able to find an event and register after brief orientation.	Should
NFR-07 Reliability	Committed data shall persist across application restarts; backup/restore steps should be documented.	Should
NFR-08 Compatibility	The UI shall support current major browsers and common mobile viewport widths.	Must
NFR-09 Maintainability	Code shall be modular, consistently named and documented.	Must
NFR-10 Data integrity	Repeated requests shall not create duplicate registrations or attendance records.	Must
NFR-11 Accessibility	Forms should have labels, keyboard access, visible focus and sufficient contrast.	Should
NFR-12 Auditability	Approval decisions and important admin changes should record actor and timestamp.	Should
NFR-13 Scalability	Architecture should allow independent API/database scaling if demand grows.	Could

6. Data Requirements

6.1 Core Entities

Entity	Indicative attributes	Purpose
Users	user_id, name, email, password_hash, role, status, created_at	Identity and access
StudentProfile	user_id, student_number, department, batch	Student-specific details
Clubs	club_id, name, description, category_id, status, advisor_user_id	Club information
ClubMembers	membership_id, club_id, user_id, status, requested_at, decided_by	Membership relationship
EventCategories	category_id, name, description	Event classification
Events	event_id, club_id, title, description, category_id, start_at, end_at, venue, capacity, deadline, status	Event details
Registrations	registration_id, event_id, user_id, status, registered_at, qr_token	Student registration
Attendance	attendance_id, registration_id, status, marked_at, marked_by, method	Attendance tracking
Certificates	certificate_id, registration_id, certificate_code, file_path, issued_at	Certificate record
Announcements	announcement_id, club_id, event_id, author_user_id, title, body, published_at	Communication
Notifications	notification_id, user_id, type, message, read_at, created_at	In-app notifications
Feedback	feedback_id, event_id, user_id, rating, comments, created_at	Optional event feedback

6.2 Data Integrity Rules

• User email shall be unique after normalization.

• Registration shall reference an existing user and event; duplicate active registrations shall be prevented.

• Membership shall reference one user and one club; a duplicate active membership should be prevented.

• Attendance shall reference a valid registration and be unique per registration.

• Certificate codes shall be unique and linked to eligible registrations.

• Referenced records should be archived or deletion-restricted to preserve history.

• Date/time values should be stored consistently, preferably UTC, and displayed in the intended local timezone.

6.3 Privacy and Retention

Only authorized roles may access identifiable participant lists. Reports should use aggregated totals where individual records are unnecessary. Retention and deletion rules should be confirmed with the university before using real student data.

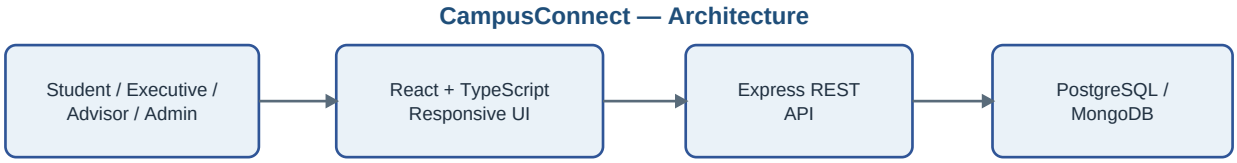
6.4 Database Selection

PostgreSQL is a suitable default because event registrations, memberships, attendance and certificates are strongly related and benefit from foreign keys, unique constraints and transactions. MongoDB is also possible, but the implementation must document how equivalent integrity rules will be enforced. Select one database for the initial release.

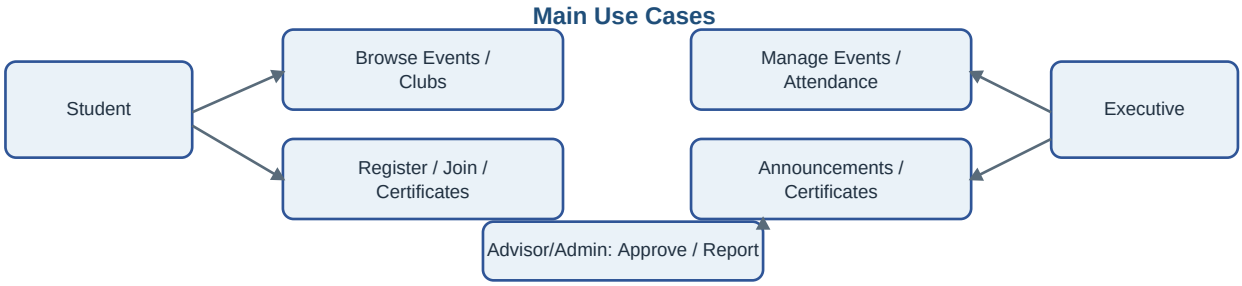
7. System Design Diagrams

These are baseline diagrams derived from the supplied project description and should be updated to match the final implementation. The original before-AI diagrams must be genuine earlier versions; this document cannot recreate files that were never provided.

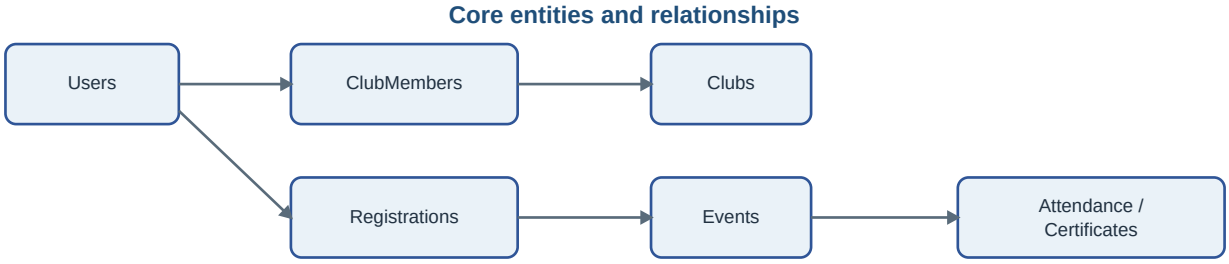
7.1 High-level Architecture



7.2 Use Case Overview



7.3 Simplified Entity Relationship Overview



Relationship notes

• One user may have many registrations; each registration belongs to one user and one event.

• One club may have many events and many members; ClubMembers resolves the many-to-many Users–Clubs relationship.

• A registration may have one attendance record and, if eligible, one certificate.

• Each event belongs to one club and may have many registrations and announcements.

1. Student opens an event. 2. Frontend requests event details. 3. Student selects Register. 4. API checks authentication, event status, deadline, capacity and duplicate registration. 5. API stores registration and creates a QR token. 6. API returns confirmation. 7. Frontend shows status and QR code. If validation fails, no registration is created and an explanatory error is returned.

7.5 Event Approval States

State	Meaning / transition
Draft	Organizer is preparing the event; Draft → Submitted.
Submitted	Awaiting review; may become Approved, Rejected or Changes Requested.
Changes Requested	Organizer revises and resubmits.
Approved	Advisor/admin has approved; event can be published.
Published	Visible to users; may become Cancelled or Completed.
Rejected	Not approved; may return to Draft if editing is allowed.
Cancelled / Completed	Event has been cancelled or completed; registration rules apply.

8. Use Cases and Workflows

UC-01: Register for an Event

Field	Description
Primary actor	Student
Preconditions	Student is authenticated; event is published and registration is open.
Main flow	Student opens event → reviews details → selects Register → API validates deadline, capacity and duplicates → registration is saved → confirmation and QR token are shown.
Alternate flow	Full/closed event, duplicate registration or invalid permissions cause rejection with a clear message.
Postconditions	One active registration exists for the student and event.

UC-02: Submit and Approve an Event

Field	Description
Actors	Club Executive; Faculty Advisor or Administrator
Preconditions	Executive is authorized for the club.
Main flow	Executive creates draft → enters required fields → submits → advisor/admin reviews → approves or requests changes → approved event is published.
Alternate flow	Missing fields block submission; rejected proposals are not visible to students.
Postconditions	Event status and decision history are saved.

UC-03: Attendance and Certificate

Field	Description
Actor	Authorized Club Executive
Preconditions	Event exists and actor has access; participant has valid registration.
Main flow	Executive scans QR or searches participant → API validates registration → attendance is recorded once → certificate is generated for eligible attendees → student downloads PDF.
Alternate flow	Invalid/reused token is rejected; manual lookup remains available to authorized staff.
Postconditions	Attendance is saved and an eligible certificate is linked to registration.

UC-04: Request Club Membership

Field	Description
Actor	Student
Preconditions	Club is active and accepting members; student is authenticated.
Main flow	Student opens club page → selects Join → system checks existing membership/request → request is saved as Pending → executive approves/rejects → student sees result.
Postconditions	Membership status is updated.

8.1 Business Rules

• Only published events appear in the public directory.

• No new registrations after the deadline or when capacity is reached.

• Executives manage only assigned clubs.

• Only authorized users may mark attendance or issue certificates.

• Certificate eligibility should be based on attendance or an explicitly configured event rule.

9. Testing and Acceptance Criteria

Maintain a separate testing report with test ID, setup, steps, expected result, actual result, pass/fail status, evidence and defect ID.

Test ID	Scenario	Expected result
TC-01	Register with valid details	Account created; duplicate email rejected.
TC-02	Login with invalid credentials	Login denied with safe error.
TC-03	Student calls admin endpoint	Forbidden/unauthorized response; no protected data disclosed.
TC-04	Register for open event	One registration and confirmation created.
TC-05	Register twice for same event	Duplicate active registration prevented.
TC-06	Register for full/closed event	Request rejected with reason.
TC-07	Executive edits another club's event	Request denied.
TC-08	Advisor approves submitted event	Decision and timestamp saved.
TC-09	Scan valid QR token	Attendance recorded against correct registration.
TC-10	Reuse invalid/used token	Attendance rejected.
TC-11	Download certificate	Eligible attendee gets PDF; unauthorized access denied.
TC-12	Use mobile viewport	Core pages usable without horizontal overflow.
TC-13	Submit invalid dates/capacity	Server-side validation rejects input.
TC-14	Unrelated user opens club report	Access denied or data correctly scoped.
TC-15	Restart app and reload records	Committed data persists.

9.1 Acceptance Criteria

• All Must requirements pass their tests or unresolved items are documented and approved by the teacher.

• Student workflow works end-to-end: browse event → register → view registration.

• Organizer workflow works end-to-end: create event → approval → publish → manage attendance.

• Backend enforces role and ownership rules, not merely hidden UI controls.

• Certificates are issued only to eligible attendees and protected from unauthorized download.

• README includes setup, environment variables, database setup, seed data and test instructions.

• Known defects and external service dependencies are listed in the final report.

10. Risks and Future Enhancements

Risk	Impact	Mitigation
Unauthorized access	Privacy/trust issues	Server-side RBAC, ownership checks, secure hashes, HTTPS and access tests
Duplicate/inconsistent records	Incorrect counts	Unique constraints, transactions and idempotent API logic
Email provider failure	Notifications delayed	In-app notices, delivery logs and retry process
QR token copied/reused	Incorrect attendance	Unpredictable token, event validation and duplicate-scan rejection
Scope creep	Late/incomplete delivery	Prioritize Must requirements; defer optional features
Database undecided	Inconsistent implementation	Choose one database and document schema decisions

10.1 Future Enhancements

• University SSO/SIS integration after approval.

• Calendar export and reminders.

• Waitlists and automated seat promotion.

• Advanced analytics and scheduled reports.

• Progressive web app/mobile enhancements.

• Multilingual UI and further accessibility work.

11. AI-Assisted Revision and Justification

This section must be completed by comparing the actual original SRS/diagrams with the revised versions. The examples below are possible justifications; use only those that accurately describe your edits. Do not claim an edit occurred unless it appears in the before/after files.

Area	Possible justification (verify against actual changes)
Requirement clarity	Broad 'manage events' statements are decomposed into testable create, approve, publish and cancel requirements.
Security	Role-based permissions and resource ownership are explicit, reducing unauthorized access risk.
Data integrity	Duplicate registration and attendance rules are defined and can be tested.
Testability	Acceptance criteria and test cases connect requirements to observable results.
Scope	Payments, mobile apps and external integrations are excluded from the initial release to keep the course project achievable.

Change log template

Section / ID	Before AI (exact original)	After AI (actual revision)	Reason / impact
[ID/section]	[Paste original text]	[Paste revised text]	[Why it changed]
[ID/section]	[Paste original text]	[Paste revised text]	[Why it changed]
[Diagram name]	[Describe original]	[Describe revised]	[What was improved]

Important: The classroom screenshot asks for both before-AI and after-AI SRS and diagrams. This PDF is the revised baseline draft only; it cannot substitute for authentic earlier versions that were not supplied.

Appendix A: Glossary

Term	Definition
Event	Scheduled university activity such as a seminar, workshop, competition or cultural program.
Club	Student organization managed by executives and supervised by an advisor.
Registration	Record of a student's intent to participate in an event.
Attendance	Record that a registered participant attended an event.
Certificate	PDF issued to a participant who meets the event's eligibility rules.
RBAC	Permission model that grants actions according to role and resource scope.
QR token	Unique token associated with a registration for attendance lookup.

Appendix B: Initial References

Consult official documentation for the selected versions and libraries:

• React documentation: https://react.dev/

• TypeScript documentation: https://www.typescriptlang.org/docs/

• Tailwind CSS documentation: https://tailwindcss.com/docs

• Node.js documentation: https://nodejs.org/docs/

• Express documentation: https://expressjs.com/

• PostgreSQL documentation: https://www.postgresql.org/docs/

• OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org/

End of document